ΕΚΘΕΣΗ ΑΝΑΛΥΣΗΣ ΠΕΡΙΣΤΑΤΙΚΟΥ (POST-MORTEM)

ΔΙΑΒΑΘΜΙΣΗ: ΔΗΜΟΣΙΑ / TLP:CLEAR

71 Μέρες στο Σκοτάδι: Πώς ένα «Αθώο» Browser Extension Έκανε Hook τη Ψηφιακή μου Ζωή (και το Ψέμα του Antivirus)

ΣΥΝΤΑΚΤΗΣ & ΕΡΕΥΝΗΤΗΣ
Christos Kataxenos (christoskataxenos.com)

ΗΜΕΡΟΜΗΝΙΑ ΟΛΟΚΛΗΡΩΣΗΣ ΕΛΕΓΧΟΥ
2026-10-31

SYSTEM / ARCHITECTURE
Chromium / Windows 11 / Debian / Android

Executive Summary: *Ανατομία ενός πραγματικού browser hijacking. 71 ημέρες αθόρυβης λειτουργίας, 1.835 μπλοκαρισμένα πακέτα από το Malwarebytes, silent DOM injections, και το πώς το Discord account μου μετατράπηκε σε stealth bot για MrBeast scam.*

`#cybersecurity` `#incident-response` `#malware-analysis` `#browser-security` `#discord` `#forensics` `#windows` `#programming-humor`

```
<div class="stat-header">Panic Level</div>
<div class="stat-num">95%</div>
<div class="stat-bar"><div class="stat-fill" style="width: 95%"></div></div>
```

```
<div class="stat-header">Freddo Espressos</div>
<div class="stat-num">18 <span class="stat-unit">cups</span></div>
<div class="stat-sub">High-octane fuel</div>
```

```
<span class="callout-icon">🚨</span>
<span class="callout-title">Το Πικρό Μάθημα</span>
```

```
<p>Το ότι το Antivirus πετάει ειδοποίηση «Malicious connection blocked»
```

δεν σημαίνει ότι είστε ασφαλείς. Σημαίνει απλώς ότι έκοψε ό,τι είχε στη μαύρη λίστα του. Όσα domains δεν υπάρχουν στη λίστα, συνεχίζουν να μιλάνε κανονικότατα με το DOM του browser σας μέσω HTTP/3 QUIC με latency 20ms.

Ως φοιτητής πληροφορικής στο ΕΑΠ και developer που περνάει ώρες με κώδικα, pointers και logs, είχα την κλασική βεβαιότητα: *«Εντάξει, ξέρω πού πατάω. Δεν ανοίγω εκτελέσιμα στην τύχη, έχω Pi-hole στο δίκτυο, τρέχω real-time προστασία, δεν κινδυνεύω».*

Στις 31 Οκτωβρίου 2026 διαπίστωσα στην πράξη πόσο λάθος έκανα.

Σκαλίζοντας τα logs και τα storage databases του συστήματός μου, ανακάλυψα ότι για **71 ολόκληρες ημέρες** (από τις 21 Αυγούστου), ο Chromium browser μου στο βασικό μου desktop PC (συγκεκριμένα ο Microsoft Edge, αν και το ζήτημα επηρεάζει οριζόντια οποιονδήποτε Chromium browser — το έλεγξα εξονυχιστικά και στο Debian Linux partition μου και στο Linux laptop μου) είχε μετατραπεί σε αθόρυβο μηχανισμό script injection. Έκανε hook σε κάθε tab που άνοιγα: από GitHub, AWS και Microsoft λογαριασμούς, μέχρι e-banking, e-commerce και το φοιτητικό portal του ΕΑΠ.

Και το εξοργιστικό; Το έκανε ενώ το antivirus στο desktop μου πετούσε καθημερινά πράσινα checkmarks ότι το σύστημα είναι προστατευμένο και χειροκροτούσε τον εαυτό του.

---

1. Το Χρονικό της Μόλυνσης: 21 Αυγούστου 2026

Στις 21 Αυγούστου κατέβασα ένα utility από το `filecr.com`. Όπως αποδείχθηκε εκ των υστέρων, η λήψη αυτή «φύτεψε» αθόρυβα στον Chromium based browser το extension **WebCRX** (`filecr-assistant`), χωρίς να ζητήσει καμία επιβεβαίωση δικαιωμάτων.

Στα ενδότερα του αρχείου `Preferences` του Edge (στο πεδίο `updateclientdata`), η καταγραφή ήταν αμείλικτη: * **Extension ID:** `ddgilliopjknmglnpkegbjpoilgachlm` (`WebCRX` / `filecr-assistant`) * **Ημερομηνία Εγκατάστασης:** 21 Αυγούστου 2026 * **Ημερομηνία Εντοπισμού & Αφαίρεσης:** 31 Οκτωβρίου 2026 * **Χρόνος Αθόρυβης Δράσης: 71 ημέρες** (επιβεβαιωμένο από τη διαφορά των εσωτερικών timestamps `dlrc` και `installdate` του browser: ακριβώς 71 ημέρες)

Την ίδια ακριβώς ημέρα εμφανίστηκε στο προφίλ και το Hola ad remover (`lalfpjdbhpmnhfofkckdpkljeilmogfl`), ενώ δύο μέρες αργότερα προστέθηκε και το CocoCut (`npdhgjfiikhgncaacnnodpfchmelnchm`).

Εδώ βρίσκεται το πρώτο εξωφρενικό στοιχείο: **Δεν εγκατέστησα ποτέ συνειδητά αυτά τα πρόσθετα.** Όταν τρέχεις κεντρικό Pi-hole και AdGuard Home σε επίπεδο δικτύου για να κόβεις διαφημίσεις και trackers πριν φτάσουν στις συσκευές, δεν έχεις κανέναν απολύτως λόγο να βάλεις ad remover extension στον browser. Μπήκαν είτε μέσω αθόρυβου bundling (side-loaded), είτε ενεργοποιήθηκαν παρασκηνιακά χωρίς καμία επιβεβαίωση.

Το extension εγκαταστάθηκε στο `Default` προφίλ. Δεν εμφάνιζε διαφημίσεις, δεν επιβάρυνε τη CPU και δεν έδινε κανένα οπτικό σημάδι. Απλώς ρύθμισε ένα background alarm με όνομα `webcrx_auto_update`, το οποίο εκτελούνταν **κάθε 30 λεπτά**.

---

2. Το Ψέμα του Antivirus: 1.835 Blocks που Έκρυβαν την Αλήθεια

Στις 23 Οκτωβρίου, το Malwarebytes άρχισε να εμφανίζει ειδοποιήσεις: > *«Malicious connection blocked: msedge.exe -> overbridgenet.com (TCP:443)»*

Στα αρχεία καταγραφής (`C:\ProgramData\Malwarebytes\MBAMService\MwacDetections`), μέτρησα **1.835 μπλοκαρισμένες εξερχόμενες συνδέσεις** προς το `overbridgenet.com` (`/jsv8/offer`), μαζί με απόπειρες του `smartscreen.exe` προς κακόβουλα endpoints.

Η πρώτη ενστικτώδης αντίδραση ήταν η ανακούφιση: *«Ωραία, το Antivirus κάνει τη δουλειά του. Μπλοκάρει τις επιθέσεις, είμαστε καλυμμένοι».*

Ήταν μια τεράστια παγίδα.

Εξετάζοντας τη βάση δικτύου του browser (`Network Persistent State`), επιβεβαίωσα ότι για το `overbridgenet.com` το SRTT (Smoothed Round-Trip Time) ήταν πράγματι `None`. Το antivirus έκοβε πράγματι το 100% των πακέτων προς *αυτό* το συγκεκριμένο domain.

Μόνο που οι επιτιθέμενοι δεν είχαν ένα μόνο domain. Είχαν πολλαπλούς C2 servers, και το antivirus γνώριζε μόνο τον έναν.

---

3. Τα 4 C2 Endpoints που Επικοινωνούσαν Ανενόχλητα

Ενώ το antivirus πανηγύριζε που έκοβε το ένα domain, άλλοι τέσσερις servers της ίδιας καμπάνιας αντάλλασσαν κανονικά δεδομένα με το extension μου μέσω **HTTP/3 QUIC**, με εξαιρετικό latency (19ms – 29ms):

Server Προορισμού	Πηγή Κίνησης	Κατάσταση	Λειτουργία
`https://rapi.acrop.net/blocklist.json`	`chrome-extension://ddgilliop...`	**ΕΠΙΤΥΧΗΣ** (SRTT: 20.9 ms)	**Κεντρικός C2:** Έστελνε δυναμικές ρυθμίσεις κάθε 30 λεπτά μέσω του alarm `webcrx_auto_update`.
`https://api.jetpusher.com`	`chrome-extension://ddgilliop...`	**ΕΠΙΤΥΧΗΣ** (SRTT: 23.2 ms)	**Telemetry Beacon:** Εκεί η επέκταση ανέφερε ζωντανά τη δραστηριότητα του browser.
`https://analytics-api.zintrack.com` & `dash.zintrack.com`	`chrome-extension://ddgilliop...`	**ΕΠΙΤΥΧΗΣ** (SRTT: 22.9 ms)	**Tracking Beacon & Push Scripts:** Σέρβιρε εξωτερικά scripts (`/js/push.js?p=2&v=1.0.2`).
`https://static-lib.com`	Μέσα από ΚΑΘΕ σελίδα!	**ΕΠΙΤΥΧΗΣ** (SRTT: 19–29 ms)	**Silent Script CDN (SCDN):** Σε κάθε tab που άνοιγα, έκανε inject 3 εξωτερικά JavaScript αρχεία στο DOM!

```
<span class="callout-icon">⚠️</span>
<span class="callout-title">Γιατί δεν χτύπησε συναγερμός;</span>
```

```
<p>Η κίνηση έφευγε από το επίσημο, ψηφιακά υπογεγραμμένο εκτελέσιμο της
```

Microsoft (`msedge.exe`) μέσω στάνταρ θυρών HTTPS/QUIC (443). Για τα εργαλεία προστασίας του λειτουργικού, επρόκειτο για συνηθισμένη περιήγηση στο διαδίκτυο.

---

4. Silent Script Injection: Το DOM στα Χέρια Τρίτων

Ψάχνοντας στον φάκελο: `C:\Users\chris\AppData\Local\Microsoft\Edge\User Data\Default\Code Cache\js` φάνηκε η πραγματική διάσταση του προβλήματος.

Μέσα σε μόλις **έξι ημέρες** (26/10 – 31/10), καταγράφηκαν **724 εκτελέσεις** τριών εξωτερικών scripts που φορτώνονταν παρασκηνιακά σε κάθε ανοιχτή ιστοσελίδα: 1. `https://static-lib.com/s/6be.../vendor.min.js?v=61.9...&r=DE&t=<JWT>` (**327 εκτελέσεις**) 2. `https://static-lib.com/l/3af.../module.core.js?lv=13&t=<JWT>` (**199 εκτελέσεις**) 3. `https://static-lib.com/s/10f.../base.min.js?v=27.9...&r=ZZ&t=<JWT>` (**198 εκτελέσεις**)

Τι μετέδιδαν αυτά τα scripts;

Αποκωδικοποιώντας τα Base64 JWT tokens (`&t=eyJ...`) και διαβάζοντας τα αρχεία LevelDB (`Local Storage\leveldb\005183.ldb`, `005187.ldb`):

- **Πλήρες Origin & Referer URL:** Το script γνώριζε ακριβώς ποια σελίδα επισκεπτόμουν.
- **Υπογεγραμμένο Session JWT:** Token ταυτοποίησης συνεδρίας με μεταδεδομένα (`{"h":"static-lib.com", "jti":"...", "iat":1793029228, ...}`).
- **Persistent Tracking IDs στο LocalStorage:** Το script έγραφε κλειδιά `__scdn_ga_sid` και `__3g4_session_id` απευθείας στο `localStorage` των σελίδων που επισκεπτόμουν (π.χ. στο `login.live.com` έγραψε `{"id":"jzxec9oyn9jn58m6j8bv1"}`), διατηρώντας το tracking ακόμα και αν άλλαζε η IP μου.
- **Άμεση Πρόσβαση στο DOM:** Επειδή ο κώδικας εκτελούνταν απευθείας μέσα στο context της κάθε ιστοσελίδας, είχε πρόσβαση σε ό,τι απεικονιζόταν στην οθόνη: φόρμες εισόδου, στοιχεία λογαριασμών και **OAuth session tokens** αποθηκευμένα στο `localStorage`.

---

5. Το Threat Model: Ποιες Υπηρεσίες Επηρεάστηκαν

Ένα script που κάνει hook στο DOM του browser εκτελείται τυφλά σε κάθε tab. Δεν ξεχωρίζει ειδησεογραφικά sites από ευαίσθητες πλατφόρμες.

Από την εξέταση των entries στο Code Cache και στο Local Storage, δεκάδες origins δέχτηκαν injection κατά την καθημερινή περιήγηση:

- **Identity Providers & Single Sign-On:** Πύλες εισόδου email, cloud λογαριασμών και code repositories. Ένα κλεμμένο session token ή OAuth token από το `localStorage` μπορεί να επιτρέψει πρόσβαση χωρίς καν να ζητηθεί 2FA.
- **Ακαδημαϊκά Portals:** Πλατφόρμες του ΕΑΠ και εκπαιδευτικά εργαλεία.
- **E-commerce:** Πλατφόρμες αγορών με αποθηκευμένες κάρτες, διευθύνσεις και ιστορικό συναλλαγών.
- **Cloud & Developer Services:** Hosting dashboards, package registries και κονσόλες διαχείρισης AI APIs.
- **Επαγγελματικά Δίκτυα:** Εργαλεία messaging, επαγγελματικές πλατφόρμες και portals καριέρας.

```
<span class="callout-icon">ℹ️</span>
<span class="callout-title">Ειρωνεία στην Πράξη</span>
```

```
<p>Το script εκτελούνταν κανονικά στο παρασκήνιο ακόμη κι όταν άνοιγα
```

σελίδες ελέγχου διαρροών (όπως το **Have I Been Pwned**) ή τα documentation portals του ίδιου του **Antivirus**. Την ώρα που έλεγχα αν είχαν διαρρεύσει παλιά δεδομένα, το injected script στο DOM κατέγραφε τη συνεδρία.

---

6. Η Πραγματική Ζημιά: Το Discord μου Μετατρέπεται σε Stealth Bot για MrBeast Scam

Μέχρι εκείνο το σημείο, θα μπορούσε κανείς να παρηγορηθεί με τη σκέψη: *«Εντάξει, ίσως ήταν απλώς ένα passive tracker ή ad-injection framework»*.

Δυστυχώς, η πραγματικότητα ήταν πολύ πιο επιθετική.

Εκείνες τις ημέρες, σε διάφορους Discord servers στους οποίους συμμετέχω, άρχισε να κυκλοφορεί το γνωστό κακόβουλο **MrBeast Scam** (ένα ψεύτικο giveaway που υπόσχεται δώρα και κάρτες για να παρασύρει ανυποψίαστους χρήστες σε phishing σελίδες).

Το απόλυτο σοκ ήρθε όταν συνειδητοποίησα ότι **ο δικός μου λογαριασμός έστελνε αυτά τα scam μηνύματα σε άλλους χρήστες χωρίς να έχω ιδέα**.

Μέσω του hook στο DOM και της πρόσβασης στα tokens του browser, οι εισβολείς είχαν αρπάξει το ενεργό session του Discord μου. Το χρησιμοποίησαν άμεσα για να μετατρέψουν τον λογαριασμό μου σε spam puppet.

Το πιο ύπουλο τέχνασμα όμως ήταν ο μηχανισμός συγκάλυψης (stealth suppression): 1. Το κακόβουλο script έστελνε το scam μήνυμα στους ανυποψίαστους χρήστες. 2. **Αμέσως μετά την αποστολή, έβαζε αυτόματα τον παραλήπτη σε Ignore / Block!**

Ο λόγος ήταν σατανικά απλός: Αν κάποιος φίλος ή μέλος του server απαντούσε έκπληκτος *«Ρε συ Χρήστο, σε χακάρανε;»* ή *«Τι link είναι αυτό που μου έστειλες;»*, το μήνυμά του δεν έφτανε ποτέ σε μένα. Δεν χτυπούσε καμία ειδοποίηση, δεν εμφανιζόταν κανένα unread badge. Το bot «φίμωνε» τα θύματα σε πραγματικό χρόνο για να μην καταλάβω ότι ο λογαριασμός μου είχε μετατραπεί σε αναμεταδότη phishing!

Αυτή ήταν η στιγμή που το περιστατικό ξέφυγε από τη θεωρία. Δεν επρόκειτο πλέον για μια απλή παραβίαση ιδιωτικότητας, αλλά για ενεργή κατάχρηση ταυτότητας (identity weaponization).

---

7. Incident Response & Home Office: Το Σενάριο του Citrix NetScaler

Τις ίδιες ακριβώς ημέρες που ερευνούσα το περιστατικό, εμφανίστηκε μια άλλη ανησυχητική συγκυρία: η σύζυγός μου εργάζεται με τηλεργασία μέσω Citrix Workspace, και η εταιρεία της απαγόρευσε ξαφνικά την εξ αποστάσεως εργασία λόγω παγκόσμιων κυβερνοεπιθέσεων σε ευπάθειες του **Citrix NetScaler**.

Μέσα στην αναστάτωση του triage, το πρώτο ερώτημα που προέκυψε ήταν αναπόφευκτο: *«Το εταιρικό της laptop βρίσκεται στο ίδιο οικιακό δίκτυο με το δικό μου μολυσμένο PC. Υπήρξε κίνδυνος μετακίνησης στο τοπικό δίκτυο (lateral movement);»*

Η τεχνική ανάλυση ξεκαθάρισε την κατάσταση:

1. **Διαφορετικά Συμβάντα (Correlation ≠ Causation):** Οι ευπάθειες του Citrix NetScaler ήταν server-side zero-days που χτυπούσαν απευθείας τα κεντρικά εταιρικά gateway appliances στο διαδίκτυο για απόσπαση session tokens από τη μνήμη. Δεν επρόκειτο για σκουλήκι στο τοπικό LAN. Η δική μου μόλυνση βρισκόταν στο PC μου ήδη από τις 21 Αυγούστου.
2. **Απομόνωση Υλικού:** Η σύζυγός μου χρησιμοποιούσε **αποκλειστικά το εταιρικό της laptop** και ποτέ το δικό μου PC. Επειδή η δική μου μόλυνση ήταν αυστηρά περιορισμένη στο context του browser (`msedge.exe`) και δεν είχε επεκταθεί στο λειτουργικό ως LAN worm, δεν υπήρχε τρόπος να επηρεαστεί το εταιρικό laptop ή τα επαγγελματικά της αρχεία.
3. **Ο Κανόνας για το Home Office:** Αν η εργασία γινόταν με όρους BYOD (Bring Your Own Device) και χρησιμοποιούνταν ο δικός μου browser για πρόσβαση στα εταιρικά συστήματα, το injected script θα είχε άμεση πρόσβαση στα εταιρικά tokens. Ο αυστηρός διαχωρισμός προσωπικού και επαγγελματικού εξοπλισμού (μαζί με απομονωμένο Guest VLAN για εταιρικές συσκευές) αποδείχθηκε σωτήριος.

---

8. Γιατί το Τοπικό DNS Sinkhole δεν Μπλόκαρε το C2

Στο σπίτι δεν βασίζομαι στο DNS του router του παρόχου. Τρέχω κεντρικό **DNS sinkhole (Pi-hole / AdGuard Home)**, failover DNS servers και διαχειριζόμενο εξοπλισμό.

Όταν το εργαλείο επιτήρησης κατέγραψε εκατοντάδες συνδέσεις προς τα κακόβουλα endpoints, το βασικό ερώτημα ήταν εύλογο: *«Πώς πέρασαν αυτά τα domains από το κεντρικό DNS φίλτρο του δικτύου;»*

Η έρευνα αποκάλυψε πέντε διακριτά κενά:

```
[ Microsoft Edge (msedge.exe) ]
        │
        ├──▶ 1. Παράκαμψη DoH (Secure DNS / DoH στον Browser) ─────────────▶ Internet (Bypass Τοπικού DNS)
        │
        ├──▶ 2. Διαρροή IPv6 (ISP DNS / Router IPv6 SLAAC) ──────────────────▶ Internet (Bypass IPv4 DNS)
        │
        └──▶ 3. Ερώτημα στο Τοπικό DNS Sinkhole
                    │
                    ├──▶ Το domain ΔΕΝ υπήρχε στις βασικές λίστες διαφημίσεων
                    │
                    └──▶ Upstream Race: Μη-φιλτραρισμένος resolver απάντησε ταχύτερα
                         από τον Security Resolver ───────────────────────────▶ Allowed (Επετράπη!)
```

1. Κενό Threat Intelligence (Ad-blocking vs Malware-blocking)

Οι απλές adlists κόβουν διαφημίσεις και banners. Ένας C2 server ή ένα dynamic tracking endpoint δεν αναγνωρίζεται ως διαφήμιση. Χωρίς εξειδικευμένα feeds threat intelligence και botnet C2, το sinkhole θεωρεί το domain καθαρό.

2. Το Upstream DNS Race Condition

Ρυθμίζω πάντα ως Upstream DNS αποκλειστικά τον **Quad9** λόγω του αυτόματου security filtering που προσφέρει. Στις ρυθμίσεις όμως του DNS sinkhole βρέθηκαν επιλεγμένοι και άλλοι, απλοί δημόσιοι resolvers (πιθανώς από παλαιότερο backup ή update).

Το αποτέλεσμα ήταν καταστροφικό: * Το sinkhole έστελνε το ερώτημα ταυτόχρονα σε όλους τους επιλεγμένους upstream servers. * Όποιος διακομιστής απαντούσε μερικά milliseconds πιο γρήγορα, κέρδιζε το race και έστελνε την απάντηση στον browser. * Ο Quad9 μπλόκαρε το domain, αλλά ο απλός resolver επέστρεφε κανονικά την πραγματική IP του C2. Αν κέρδιζε ο απλός resolver, η προστασία ακυρωνόταν πλήρως.

3. Διαρροή IPv6

Το router μοίραζε IPv6 διευθύνσεις μέσω SLAAC και διαφήμιζε τον δικό του DNS. Τα Windows προτιμούν το IPv6 από προεπιλογή, οπότε πολλά ερωτήματα παρέκαμπταν εντελώς το τοπικό IPv4 sinkhole.

4. DoH στον Browser: Παράκαμψη της Τοπικής Ασφάλειας

Οι σύγχρονοι browsers προωθούν το **DNS-over-HTTPS (DoH)** ως χαρακτηριστικό ιδιωτικότητας.

Στην πράξη, αν δεν ελεγχθεί κεντρικά: 1. Αγνοεί τις ρυθμίσεις DNS του λειτουργικού συστήματος και του τοπικού δικτύου. 2. Πακετάρει τα DNS queries σε κανονικό HTTPS traffic (Port 443) προς δημόσιους παρόχους. 3. Ακυρώνει τα τοπικά Pi-hole και AdGuard φίλτρα, επιτρέποντας σε επεκτάσεις και scripts να επικοινωνούν με εξωτερικούς servers χωρίς κανέναν έλεγχο.

Αν έχετε τοπικό DNS blocker: **Απενεργοποιήστε το "Secure DNS" στον browser** ή ορίστε ρητά ως DoH endpoint τον δικό σας τοπικό server.

5. Διαφορά Layer 3 (IP) και Layer 7 (DNS)

Ακόμη κι αν ένα domain κοπεί στο DNS, αν η εφαρμογή έχει αποθηκεύσει την IP στην cache (`ipconfig /displaydns`) ή αν το script χρησιμοποιεί απευθείας IP, το DNS μπλοκάρισμα δεν αρκεί. Απαιτείται **Firewall σε Επίπεδο 3/4** στο ίδιο το λειτουργικό σύστημα.

Η Αναβάθμιση του Δικτύου

Για την αντιμετώπιση αυτών των κενών: 1. **Εξειδικευμένα Feeds:** Προστέθηκαν λίστες Threat Intelligence, C2 και Anti-Malware, υπερτριπλασιάζοντας την κάλυψη. 2. **Καθαρό Upstream DNS:** Αφαιρέθηκαν όλοι οι μη-φιλτραρισμένοι resolvers. Παρέμειναν μόνο resolvers με υποστήριξη DNSSEC και malware blocking. 3. **Failover Redundancy:** Δημιουργία δεύτερου ανεξάρτητου DNS sinkhole για αδιάλειπτη διαθεσιμότητα. 4. **Endpoint Firewall:** Άμεσος αποκλεισμός των εντοπισμένων C2 IPs στο Windows Defender Firewall.

Τομέας Ασφαλείας	Πριν το Συμβάν	Μετά τη Θωράκιση
Κάλυψη DNS Λιστών	Μόνο γενικές διαφημίσεις & trackers	Threat Intelligence, C2 Botnets & Anti-Malware
Κακόβουλα C2 Domains	❌ Απροστάτευτο σε επίπεδο DNS	✅ Πλήρης Wildcard Αποκλεισμός
Upstream DNS Resolution	❌ Race Condition με unfiltered resolvers	✅ Αυστηρά Security Resolvers (DNSSEC + Malware Filtering)
Διαθεσιμότητα DNS (Failover)	❌ Μοναδικό σημείο αποτυχίας (SPOF)	✅ Πλήρως πλεονάζουσα υποδομή (Primary + Secondary Failover)
Προστασία IP (Layer 3)	⚠️ Καμία προστασία σε direct IP calls	✅ Αυτόματοι κανόνες Firewall στο Endpoint
Επιτήρηση Δικτύου	⚠️ Παθητικό ad-blocking	✅ Ενεργή αποτροπή απειλών πολλαπλών επιπέδων

---

9. Συστηματικός Έλεγχος του Συστήματος

Πριν προχωρήσω σε επανεγκατάσταση του λειτουργικού, πραγματοποίησα έλεγχο σε όλα τα υποσυστήματα:

```
<div class="terminal-dots">
  <span class="t-dot red"></span>
  <span class="t-dot yellow"></span>
  <span class="t-dot green"></span>
</div>
<div class="terminal-title">PowerShell Forensic Investigation</div>
```

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-PowerShell/Operational'; Id=4104} |
    Select-Object -First 500 |
    Where-Object { $_.Message -match 'DownloadString|WebClient|IEX|Invoke-RestMethod' }
# Result: 0 suspicious outbound network calls detected.

Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Bits-Client/Operational'; Id=3,4,59,60}
# Result: Only legitimate Microsoft OneDrive & EdgeWebView2 updates.
```

- **PowerShell Logs (Event ID 4104):** Έλεγχος σε 500 καταγεγραμμένα script blocks. Κανένα δεν εμφάνισε ύποπτες δικτυακές κλήσεις.
- **Windows BITS Logs:** Όλες οι μεταφορές αφορούσαν νόμιμες ενημερώσεις λογισμικού.
- **Το αρχείο `%TEMP%\start.js` (`Worm.Nemucod.JS`):** Εντοπίστηκε ανενεργό script 85 bytes από τις 28/10 στο `%TEMP%`, χωρίς ενδείξεις εκτέλεσης ή επικοινωνίας.
- **Cross-Platform Έλεγχος (Debian Linux & Laptop):** Στο desktop PC διατηρώ dual-boot εγκατάσταση με σύστημα βασισμένο σε **Debian Linux**, ενώ χρησιμοποιώ και Linux laptop. Έλεγξα εξονυχιστικά όλα τα προφίλ Chromium, Brave και Google Chrome σε όλες τις πλατφόρμες. Ήταν **100% καθαρά** — η μόλυνση περιορίστηκε αυστηρά στο συγκεκριμένο default προφίλ του Edge.

---

10. Τα Άμεσα Βήματα του Incident Response

Σε περιστατικά παραβίασης browser ή διαρροής tokens, αυτή είναι η σειρά ενεργειών:

Βήμα 1: Άμεση Απομόνωση

1. **Αποσύνδεση από το δίκτυο** (αφαίρεση Ethernet ή απενεργοποίηση Wi-Fi).
2. **Καμία εισαγωγή κωδικών από το μολυσμένο σύστημα.**

Βήμα 2: Αλλαγή Κωδικών από Καθαρή Συσκευή

Με προτεραιότητα: 1. **Primary Email & Identity Providers** (Google, Microsoft) 2. **Cloud & Developer Accounts** (GitHub, AWS, Cloudflare) 3. **Οικονομικές Υπηρεσίες** (PayPal, e-banking) 4. **Επικοινωνία & Social** (Discord, LinkedIn)

Βήμα 3: Ανάκληση Ενεργών Sessions (Session Revocation)

```
<span class="callout-icon">🚨</span>
<span class="callout-title">Το 2FA δεν ακυρώνει ενεργό session cookie</span>
```

```
<p>Η αλλαγή κωδικού πρόσβασης δεν ακυρώνει απαραίτητα τα ήδη εκδοθέντα
```

session cookies ή OAuth tokens. Πρέπει να επιλέξετε **«Αποσύνδεση από όλες τις συσκευές / Revoke all sessions»** στις ρυθμίσεις ασφαλείας κάθε υπηρεσίας.

Βήμα 4: Εκκαθάριση Browser

1. Αφαίρεση όλων των μη απαραίτητων ή ύποπτων επεκτάσεων.
2. Πλήρης διαγραφή Cookies, Local Storage, Cache και Service Workers.
3. Έλεγχος των καταλόγων:
   - `%LocalAppData%\Microsoft\Edge\User Data\Default\Extensions`
   - `%LocalAppData%\Microsoft\Edge\User Data\Default\Local Storage\leveldb`
   - `%LocalAppData%\Microsoft\Edge\User Data\Default\Code Cache`

Βήμα 5: Hardening στα Windows

Απενεργοποίηση του Windows Script Host για τον τρέχοντα χρήστη, ώστε να μπλοκάρεται η αυτόματη εκτέλεση αρχείων `.vbs` και `.js`:

```
# Απενεργοποίηση Windows Script Host για τον τρέχοντα χρήστη
$wshPath = "HKCU:\Software\Microsoft\Windows Script Host\Settings"
if (-not (Test-Path $wshPath)) {
    New-Item -Path $wshPath -Force | Out-Null
}
Set-ItemProperty -Path $wshPath -Name "Enabled" -Value 0 -Type DWord
```

---

11. The Aftermath: Όταν η Παράνοια Γίνεται Κώδικας (Project Sotiria)

Στις 3:30 τα ξημερώματα, αφού έχεις ολοκληρώσει όλα τα βήματα του incident response —έχεις κόψει το δίκτυο, έχεις αλλάξει κωδικούς, έχεις ακυρώσει όλα τα sessions— μένει μια επίμονη αμφιβολία. Τα εμπορικά εργαλεία απέτυχαν: το antivirus έδινε ψευδή αίσθηση ασφάλειας με πράσινα τικ ενώ γινόταν data exfiltration, και ο browser άνοιγε DoH τούνελ στο παρασκήνιο.

Και το βλέμμα πέφτει στο Android κινητό δίπλα στο πληκτρολόγιο. Εκεί όπου είναι συνδεδεμένα τα ίδια Google accounts, τρέχουν τα authenticators και συγχρονίζονται tabs, bookmarks και passwords. Αν ένας εισβολέας είχε αθόρυβη πρόσβαση στο DOM του βασικού σου browser επί 71 ημέρες, δεν μπορείς να θεωρείς τίποτα δεδομένο.

Εκείνο το βράδυ γεννήθηκε η **Sotiria (Sotiria — Endpoint Sentinel)** — από την ελληνική λέξη *Σωτηρία*: ένα αυτόνομο αμυντικό σύστημα για Windows (με ειδικό audit pipeline για Android), σχεδιασμένο να εντοπίζει droppers, stealers και μηχανισμούς persistence που τα παραδοσιακά εργαλεία προσπερνούν.

Χωρίς να αποκαλύψω ακόμα ολόκληρη την αρχιτεκτονική της, η Sotiria συνδυάζει: * **Χειρουργική Απομόνωση & YARA Hunting:** Σάρωση ύποπτων patterns σε staging φακέλους και άμεση μεταφορά σε κρυπτογραφημένο Quarantine Vault — χωρίς τυφλές διαγραφές. * **Aggressive Hardening & Persistence Neutralization:** Εξουδετέρωση κακόβουλων Scheduled Tasks και Registry keys, πλήρης απενεργοποίηση του Windows Script Host και κλείδωμα δικαιωμάτων φακέλων. * **Live Process Watchdog & Dynamic Firewall Shield:** Παρακολούθηση διεργασιών για εκτελέσεις από ύποπτα paths και αυτόματο μπλοκάρισμα C2 διευθύνσεων στο Windows Firewall σε πραγματικό χρόνο. * **Mobile Security Audit μέσω ADB:** Read-only επιθεώρηση του συνδεδεμένου Android για ύποπτα Accessibility Services, κρυφά sideloaded APKs και πιθανούς keyloggers. * **Dark-SOC Console:** Ένα custom .NET 8 dashboard με live μετρήσεις, real-time logs και εξαγωγή επίσημων audit reports.

Αυτό το σύστημα μου επέτρεψε να κλείσω τον υπολογιστή με ασφάλεια. Η πλήρης τεχνική του ανάλυση, ο κώδικας των modules και το στήσιμό του όμως ανήκουν σε δικό τους άρθρο.

```
<span class="callout-icon">🎯</span>
<span class="callout-title">Coming Up Next: Part 2 — Project Sotiria</span>
```

```
<p>Στο επόμενο άρθρο, περνάμε από το Incident Response στην
```

Ενεργητική Άμυνα: Πλήρης τεχνική ανάλυση της **Sotiria (Endpoint Sentinel)** — η αρχιτεκτονική της, τα source modules, το GUI και πώς μπορείτε να χτίσετε το δικό σας σύστημα προστασίας για Windows και Android όταν τα εμπορικά εργαλεία αποτυγχάνουν. *Stay tuned!*

Συμπέρασμα

Όταν ηρεμεί η αδρεναλίνη και τα logs επιβεβαιώνουν ότι οι συνδέσεις κόπηκαν, μένουν τρία βασικά συμπεράσματα:

1. **Τα Browser Extensions έχουν υπερβολικά δικαιώματα:** Μια επέκταση με άδεια ανάγνωσης και τροποποίησης δεδομένων σε όλες τις ιστοσελίδες έχει ουσιαστικά μεγαλύτερη πρόσβαση στα προσωπικά σας δεδομένα από ένα ανεξάρτητο πρόγραμμα. Εκτελείται μέσα στο authenticated session σας και παρακάμπτει τα παραδοσιακά εργαλεία προστασίας.
2. **Μην επαναπαύεστε στα μπλοκαρισμένα πακέτα:** Το ότι το antivirus σταμάτησε 1.835 συνδέσεις προς ένα domain δεν σήμαινε τίποτα, όσο άλλα τέσσερα domains επικοινωνούσαν ελεύθερα στο παρασκήνιο.
3. **Άμυνα σε βάθος (Defense in Depth):** Χρήση password manager με μοναδικούς κωδικούς, υποχρεωτικό 2FA με Hardware Key ή Authenticator app, τακτικός έλεγχος των επεκτάσεων και άμεση ανάκληση ενεργών sessions στο παραμικρό ύποπτο σύμπτωμα.

Τουλάχιστον, μέσα σε όλη την ταλαιπωρία, όποιος ανέλυε την κίνηση από το portal του ΕΑΠ θα διαπίστωσε από πρώτο χέρι τις δυσκολίες των διακριτών μαθηματικών της **ΠΛΗ12**. Κάτι είναι κι αυτό.

Ελέγξτε τις επεκτάσεις στον browser σας σήμερα — και μην περιμένετε από το antivirus να σας ειδοποιήσει.